Transaction Processing

Department of Computer Science & Engineering

Loknayak Jai Prakash Institute of Technology Chapra

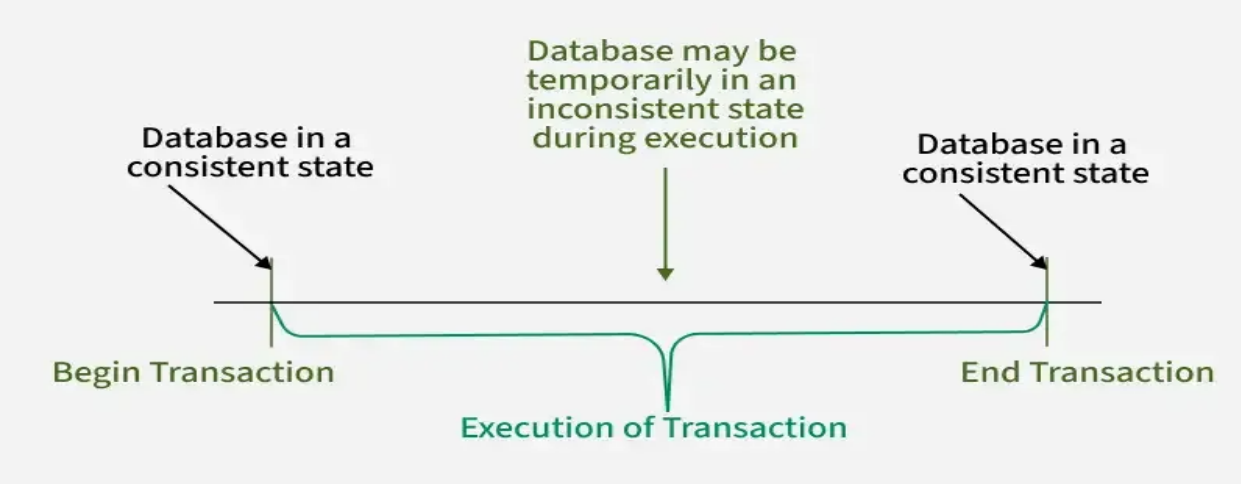
Presented by

Soniya Rohhila

Assistant Professor (CSE)

Transaction Concept

- A transaction refers to a sequence of one or more operations (such as read, write, update, or delete) performed on the database as a single logical unit of work.
- A transaction ensures that either all the operations are successfully executed (committed) or none of them take effect (rolled back).
- Two main issues to deal with:
  - Failures of various kinds, such as hardware failures and system crashes
  - Concurrent execution of multiple transactions



Operations of Transaction

- **Read(X) -** read the value of a particular database element X and stores it in a temporary buffer in the main memory for further actions
  - Example: For a banking system, when a user checks their balance, a Read operation is performed on their account balance
- **Write(X)** - stores updated data from main memory back to the database
  - Example: if a user withdraws money, a Write operation is performed after the balance is updated
- **Commit** - used to maintain integrity in the database. Due to some failure of power, hardware, or software, etc., a transaction might get interrupted before all its operations are completed. This may cause ambiguity in the database, i.e. it might get inconsistent before and after the transaction.
  - Example: After a successful money transfer in a banking system, a Commit operation finalizes the transaction
- **Rollback** - undoes all changes made by a transaction if an error occurs, restoring the database to its last consistent state.
  - Suppose system encounters an issue, like insufficient funds in the sender's account. In that case, the transaction is rolled back

ACID Properties of Transaction

Transactions in DBMS must ensure data is accurate and reliable. They follow four key ACID properties:

- **Atomicity:** A transaction is all or nothing. If any part fails, the entire transaction is rolled back. Example: While transferring money, both debit and credit must succeed. If one fails, nothing should change.
- **Consistency:** A transaction must keep the database in a valid state, moving it from one consistent state to another. Example: If balance is ₹1000 and ₹200 is withdrawn, the new balance should be ₹800.
- **Isolation:** Transactions run independently. One transaction's operations should not affect another's intermediate steps. Example: Two users withdrawing from the same account must not interfere with each other's balance updates.
- **Durability:** Once a transaction is committed, its changes stay even if the system crashes. Example: After a successful transfer, the updated balance remains safe despite a power failure.

ACID Properties of Transaction

Atomicity

- It states that all operations of the transaction take place at once if not, the transaction is aborted.
- There is no midway, i.e., the transaction cannot occur partially. Each transaction is treated as one unit and either run to completion or is not executed at all.
- Atomicity involves the following two operations:
- **Abort:** If a transaction aborts then all the changes made are not visible.
- **Commit:** If a transaction commits then all the changes made are visible.

ACID Properties of Transaction

- **Example:** Let's assume that following transaction T consisting of T1 and T2. A consists of Rs 600 and B consists of Rs 300. Transfer Rs 100 from account A to account B.

After completion of the transaction, A consists of Rs 500 and B consists of Rs 400.

- If the transaction T fails after the completion of transaction T1 but before completion of transaction T2, then the amount will be deducted from A but not added to B. This shows the inconsistent database state. In order to ensure correctness of database state, the transaction must be executed in entirety.

ACID Properties of Transaction

T1=complete HDFC	T2=complete SBI
Read(A)=600 A:= A-100 Write(A) = 500	Read(B)=300 Y:= Y+100 Write(B)=400

Consistency

- The integrity constraints are maintained so that the database is consistent before and after the transaction.
- The execution of a transaction will leave a database in either its prior stable state or a new stable state.
- The consistent property of database states that every transaction sees a consistent database instance.
- The transaction is used to transform the database from one consistent state to another consistent state.

ACID Properties of Transaction

For example: The total amount must be maintained before or after the transaction.

1. Total before T occurs = 600+300=900
2. Total after T occurs= 500+400=900

Therefore, the database is consistent. In the case when T1 is completed but T2 fails, then inconsistency will occur.

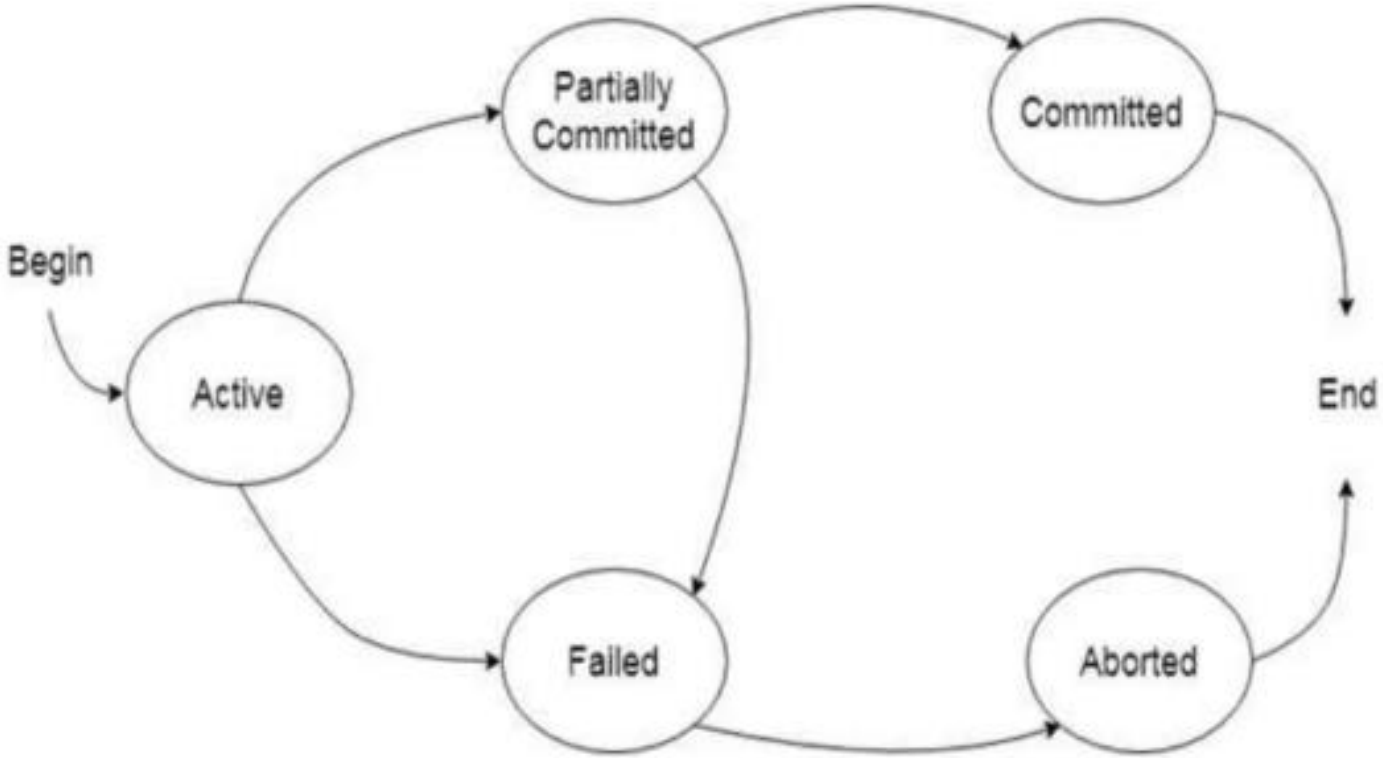
Isolation

- It shows that the data which is used at the time of execution of a transaction cannot be used by the second transaction until the first one is completed.
- In isolation, if the transaction T1 is being executed and using the data item X, then that data item can't be accessed by any other transaction T2 until the transaction T1 ends.
- The concurrency control subsystem of the DBMS enforced the isolation property.

Durability

- The durability property is used to indicate the performance of the database's consistent state. It states that the transaction made the permanent changes.
- They cannot be lost by the erroneous operation of a faulty transaction or by the system failure. When a transaction is completed, then the database reaches a state known as the consistent state. That consistent state cannot be lost, even in the event of a system's failure.
- The recovery subsystem of the DBMS has the responsibility of Durability property.

States of Transaction



States of Transaction

Active state

- The active state is the first state of every transaction. In this state, the transaction is being executed.
- For example: Insertion or deletion or updating a record is done here. But all the records are still not saved to the database.

Partially committed

- In the partially committed state, a transaction executes its final operation, but the data is still not saved to the database.
- In the total mark calculation example, a final display of the total marks step is executed in this state.

Committed

- A transaction is said to be in a committed state if it executes all its operations successfully. In this state, all the effects are now permanently saved on the database system.

Failed state

- If any of the checks made by the database recovery system fails, then the transaction is said to be in the failed state.
- In the example of total mark calculation, if the database is not able to fire a query to fetch the marks, then the transaction will fail to execute.

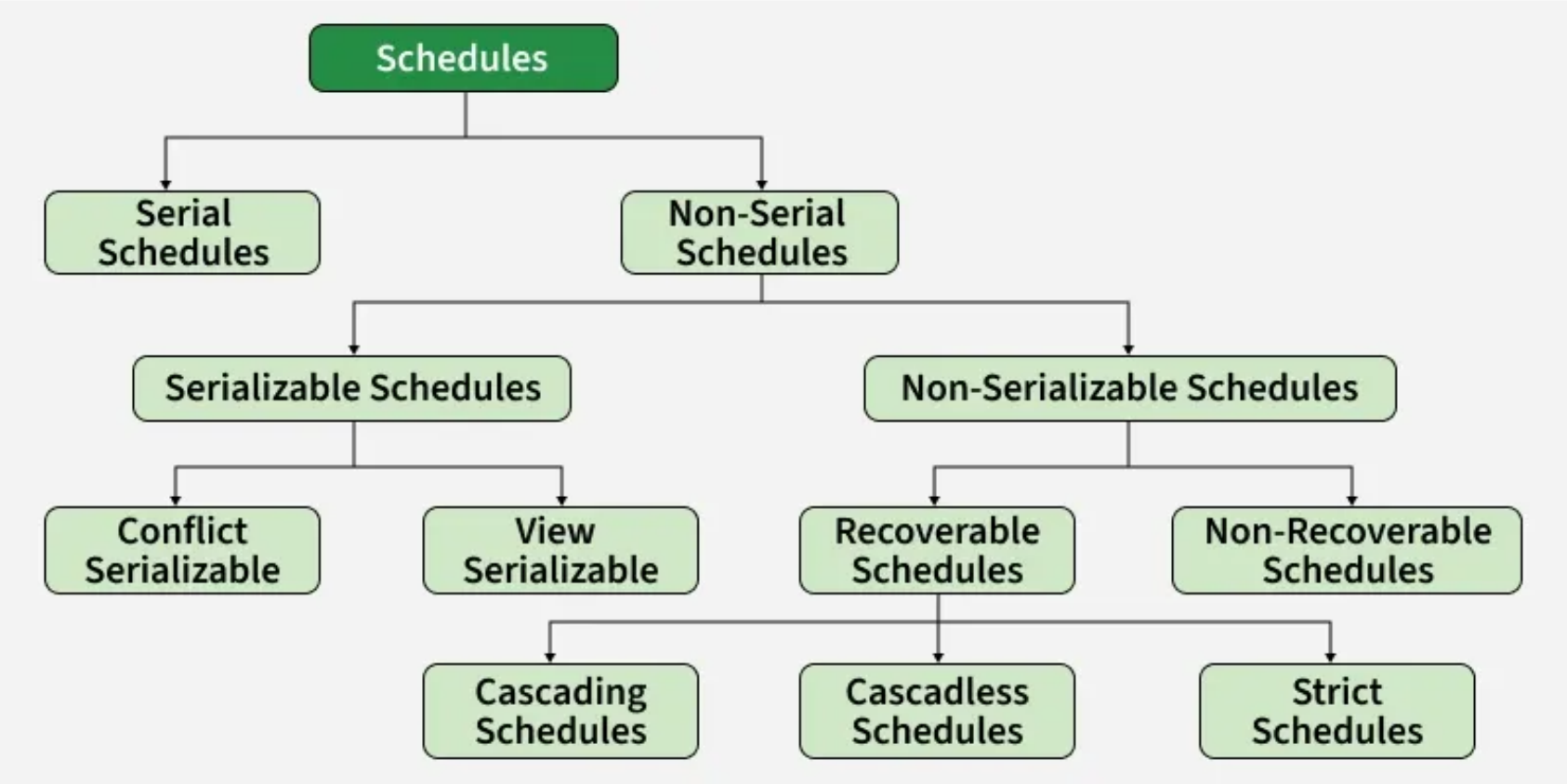
Aborted

- If any of the checks fail and the transaction has reached a failed state then the database recovery system will make sure that the database is in its previous consistent state. If not then it will abort or roll back the transaction to bring the database into a consistent state.
- If the transaction fails in the middle of the transaction then before executing the transaction, all the executed transactions are rolled back to its consistent state.
- After aborting the transaction, the database recovery module will select one of the two operations:
  1. Re-start the transaction
  2. Kill the transaction

Schedule

- A series of operation from one transaction to another transaction is known as schedule. It is used to preserve the order of the operation in each of the individual transaction.

Transaction Schedule



Serial Schedule

- The serial schedule is a type of schedule where one transaction is executed completely before starting another transaction. In the serial schedule, when the first transaction completes its cycle, then the next transaction is executed.
- **For example:** Suppose there are two transactions T1 and T2 which have some operations. If it has no interleaving of operations, then there are the following two possible outcomes:

1. Execute all the operations of T1 which was followed by all the operations of T2.
2. Execute all the operations of T1 which was followed by all the operations of T2.

- In the given (a) figure, Schedule A shows the serial schedule where T1 followed by T2.
- In the given (b) figure, Schedule B shows the serial schedule where T2 followed by T1.

In the given (a) figure, Schedule A shows the serial schedule where T1 followed by T2.

(a)

Time ↓

T1	T2

read(A);
A := A − N;
write(A);
read(B);
B := B + N;
write(B);	

	read(A);
	A := A + M;
	write(A);

Schedule A

Transaction Schedule

(b)

Time

T1	T2

	read(A);
	A := A + M;
	write(A);

read(A);
A:= A - N;
write(A);
read(B);
B:= B +N;
write(B);	

Schedule B

Non-serial Schedule

- If interleaving of operations is allowed, then there will be non-serial schedule.
- It contains many possible orders in which the system can execute the individual operations of the transactions.
- In the given figure (c) and (d), Schedule C and Schedule D are the non-serial schedules. It has interleaving of operations.

Transaction Schedule

(c)

Time

T1	T2
read(A); A := A – N;	
	read(A); A := A + M;
write(A); read(B);	
	write(A);
B := B + N; write(B);	

Schedule C

Transaction Schedule

(d)

Time

T1	T2

read(A);
A := A – N;
write(A);	

	read(A);
	A := A + M;
	write(A);

read(B);
B := B + N;
write(B);	

Schedule D

Serializable schedule

- The serializability of schedules is used to find non-serial schedules that allow the transaction to execute concurrently without interfering with one another.
- It identifies which schedules are correct when executions of the transaction have interleaving of their operations.
- A non-serial schedule will be serializable if its result is equal to the result of its transactions executed serially.

Transaction Schedule

Time ↓

T1	T2	T3
Read(A)		
	Read(B)	
A:= f_1(A)		
		Read(C)
	B:= f_2(B)	
	Write(B)	
		C:= f_3(C)
		Write(C)
Write(A)		
		Read(B)
	Read(A)	
	A:= f_4(A)	
Read(C)		
	Write(A)	
C:= f_5(C)		
Write(C)		
		B:= f_6(B)
		Write(B)

Schedule S1

Thank You